Voice of Associates (VOA) Console

People Analytics & Engagement Action Dashboard

Amrit Raj Biswal
Tech Mahindra People Analytics & Systems Group

July 13, 2026

Abstract

The **Voice of Associates (VOA)** Console is an enterprise-grade analytics dashboard designed to ingest, clean, normalize, and analyze employee connect sessions, priority risk items, and organizational action plans. It enables HR Business Partners (HRBPs) to identify attrition risks early via a composite, mathematically modeled Risk Score, surface key organizational themes using the proprietary CARES framework, and leverage generative AI for executive briefings. This document details the system's architecture, data pipeline, color-theoretic UI, and algorithms.

`Repository Path: c:\Users\profe\Downloads\HR P`

Contents

1 Executive Summary & Value Proposition

In modern organizational structures, employee feedback is generated across dozens of distinct channels and formats. Collecting, normalising, and reacting to this feedback remains a critical blocker for retention. The Voice of Associates (VOA) Console resolves this bottleneck through the following mechanisms:

- **Real-Time Attrition Risk Mapping**: Computes a dynamic composite **Risk Score (0–100)** for every associate connect to highlight high-priority items.
- **Thematic Trend Grouping**: Automatically extracts unstructured text feedback and maps it against the five pillars of the **CARES Framework** (Alignment, Career, Empowerment, Recognition, Strive).
- **Action Center Accountability**: Flattens and tracks assignments, ensuring HR owners resolve outstanding tasks inside a Kanban-style structure.
- **Dynamic Executive Briefings**: Automatically builds data payloads for LLM APIs (OpenAI or Gemini) to generate five-bullet summaries for quick leadership reviews.

2 System Architecture & Data Flow

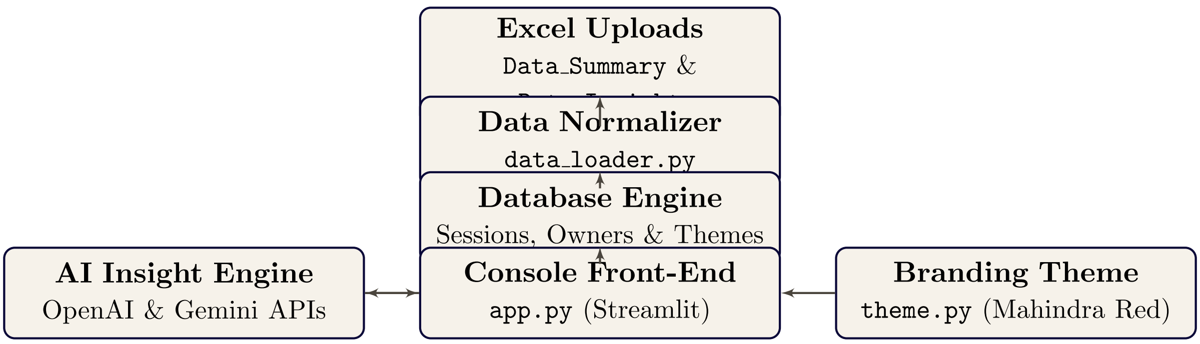
The application structure is built on a clean three-tiered design: an Ingestion & Processing Engine, a custom Design Theme utility, and a Streamlit UI Console.

2.1 Core Components

1. `app.py`: The orchestrator of the web UI. Handles Streamlit session states, multi-page routing (6 tabs), dynamic filter queries, and API connections to LLMs.
2. `data`$_loader.py : Thedata-wranglingengine.Featuresregex-basedcolumnnormalization, relationaldoul$
$lookupjoins, andafallbackhigh - fidelitysyntheticdemogenerator.$
2. `theme.py: The branding system. Declares color tokens, overrides default Plotly layout structures, and manages DOM styling using injected CSS and MutationObserver Javascript scripts.`

2.2 Architectural Layout

The diagram below illustrates the ingestion-to-visualization path:



3 Data Normalization & Ingestion Pipeline

To support files edited by different teams, the ingestion script uses regex patterns to reconcile column keys.

3.1 Fuzzy Mapping Rules

The loader applies a compiled list of regular expressions to inspect columns:

- **Associate Name**: matches `\bname\b`, `associate.*name`, `employee.*name`.
- **CARES Framework Lever**: matches `cares.*framework`, `cares.*lever`, `cares`, `framework`.
- **Action Owners**: matches `action.*owner`, `owner.*action`, `assigned.*to`, `owners`.
- **Observations**: matches `coverage`, `item.*coverage`, `theme`, `insight`.

3.2 Relational Normalization

The process decomposes the uploaded workbook into a normalized format:

1. **Artificial Keying**: Excel rows are mapped to a strict primary key `ID` range $(1, 2, 3 \ldots n)$.
2. **Compound Lookup Join**: Observation records in sheet `Data_Insight` are mapped back to parent records using a double-index lookup matching `(BHR Employee ID/Name, Date)` or the sheet row index as a fallback.
3. **Long-Format Decomposition**: Multivalue list fields (like multiple action owners separated by semicolons) are exploded into separate indices (`owners_long`) to prevent duplicate rows.

4 Mathematical Modeling: Composite Risk Score

Each associate connect session is assigned a composite risk score to measure urgency.

4.1 The Formula

The risk metric is computed as a weighted summation:

$$R = W_{\text{severity}} + W_{\text{status}} + W_{\text{sentiment}} + W_{\text{age}} \tag{1}$$

where the maximum potential score is $R_{\max} = 100$.

4.2 Weight Assigments

1. **Severity Weight** ($W_{\text{severity}} \in \{6, 22, 45\}$):

$$W_{\text{severity}} = \begin{cases} 45 & \text{if High/Critical} \\ 22 & \text{if Moderate/Medium} \\ 6 & \text{if Low/Minor} \\ 10 & \text{otherwise} \end{cases} \tag{2}$$

2. **Status Weight** ($W_{\text{status}} \in \{0, 14, 30\}$):

$$W_{\text{status}} = \begin{cases} 30 & \text{if Open/New} \\ 14 & \text{if In-progress/Active} \\ 0 & \text{if Closed/Resolved} \end{cases} \tag{3}$$

3. **Sentiment Weight** ($W_{\text{sentiment}} \in \{0, 3, 8, 15\}$):

$$W_{\text{sentiment}} = \begin{cases} 15 & \text{if Negative/Poor} \\ 8 & \text{if Mixed} \\ 3 & \text{if Neutral} \\ 0 & \text{if Positive} \end{cases} \tag{4}$$

4. **Age Penalty Weight** ($W_{\text{age}} \in [0, 10]$): Calculated only for unresolved statuses (Open or In-progress) to track compliance:

$$W_{\text{age}} = \begin{cases} \min\left(10, \dfrac{\text{Days Since Connect}}{10}\right) & \text{if Status } \neq \text{Closed} \\ 0 & \text{if Status } = \text{Closed} \end{cases} \tag{5}$$

5 Design System Guidelines

The styling definitions implemented in `theme.py` follow Tech Mahindra's official layout guidelines, employing a strict **60-30-10 color split**:

Table 1: Brand Styling Parameters

Proportion	Hex Code	UI Element / Architectural Purpose
60% Warm Neutrals	`#F6F2EA` / `#FFFFFF`	Core application canvas, main panel grids, and card backgrounds.
30% Anchors	`#0A0838` / `#4A453D`	Headers, navigation blocks, and sidebar control panels.
10% Mahindra Red	`#E31837`	High-priority metrics, warning messages, and active button highlights.

5.1 Typography Guidelines

- **Display Font**: *Newsreader* (Serif) is used for headers to establish an elegant, premium look.
- **Body Font**: *Inter* (Sans-serif) handles interface copy, tags, and charts.
- **Monospace Font**: *IBM Plex Mono* renders numeric figures, date indicators, and code values.

6 Implementation & Compilation

To build and launch the console environment locally, configure the prerequisites:

6.1 Prerequisites Installation

The application runs on Python 3.8+. Install required dependencies:

```
pip install streamlit pandas plotly openpyxl
```

6.2 Server Inception

Launch the web app directly from the terminal root:

```
cd "C:\Users\profe\Downloads\HR P"
streamlit run app.py
```